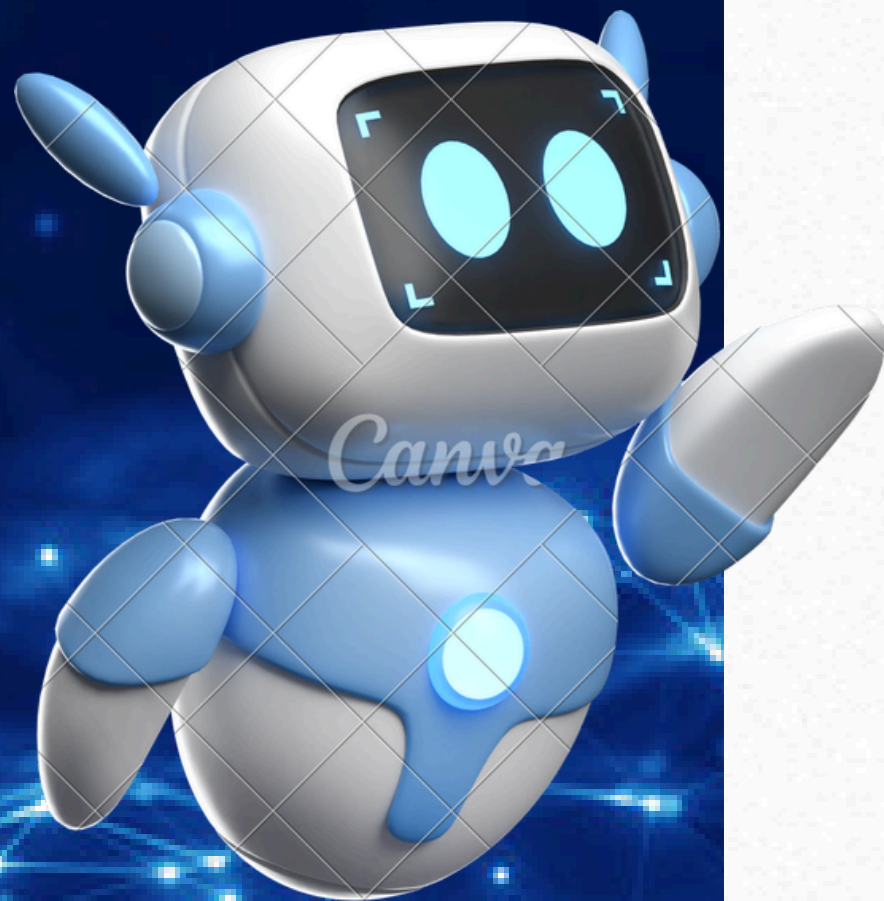




Sesión 2

Python para el Análisis de Datos

Bernick Salvador – Data Scientist Advanced Prima AFP

“Python para el Análisis de Datos”



 **Bernick Salvador**
Data Scientist Advanced

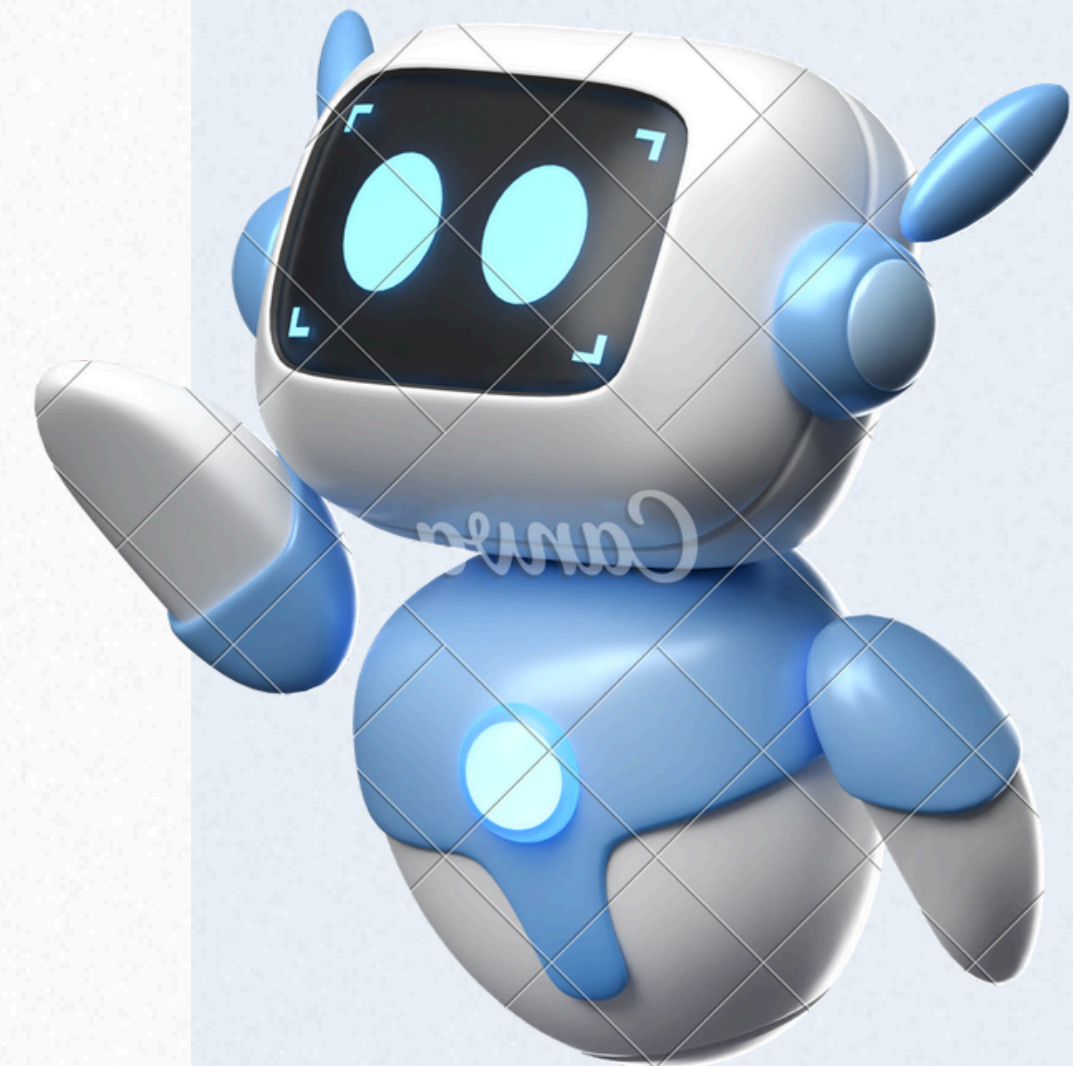
Bachiller en Ingeniería Electrónica (UNI) con estudios de posgrado en Procesamiento Digital de Señales e Imágenes, Ciencias de la Computación (UNI), Estadística en Investigación (UPCH) y Gerencia de Sistema y TI (UDH) . Actualmente sigue una especialización MBA en Ciencia de Datos en la Universidad de Sao Paulo (USP) - Brasil.

~6 años de experiencia en el mundo de la ciencia de datos laborando en empresas e instituciones como Agencia Cooperación Alemana (GIZ Perú) , Ministerio de Educación (MINEDU) e Inetum Perú.

2 años dictando cursos sobre programación y desarrollo de modelos de ML y DL en Escuela Global, CIIP Latam y DSRP

Temario de la sesión

- »»» Conceptos básicos fundamentales de Python
- »»» Numpy: arrays, indexing, slicing, iterating
- »»» Funciones y métodos de Numpy
- »»» Generación de números aleatorios
- »»» Exploración de datasets públicos
- »»» Lectura de archivos con Numpy



Declaración de **variables** en Python

- Las variables son contenedores para almacenar valores de datos.
- Python no tiene ningún comando para declarar una variable.
- Una variable se crea en el momento en que le asigna un valor por primera vez.
- No es necesario declarar las variables con ningún tipo en particular, e incluso pueden cambiar de tipo después de que se hayan establecido.

```
In [ ]: numero_entero = 4
```

```
In [ ]: mensaje = "Hola a todos!"
```


Tipos de datos en Python

Python tiene los siguientes **tipos de datos** integrados de forma predeterminada, en estas categorías:

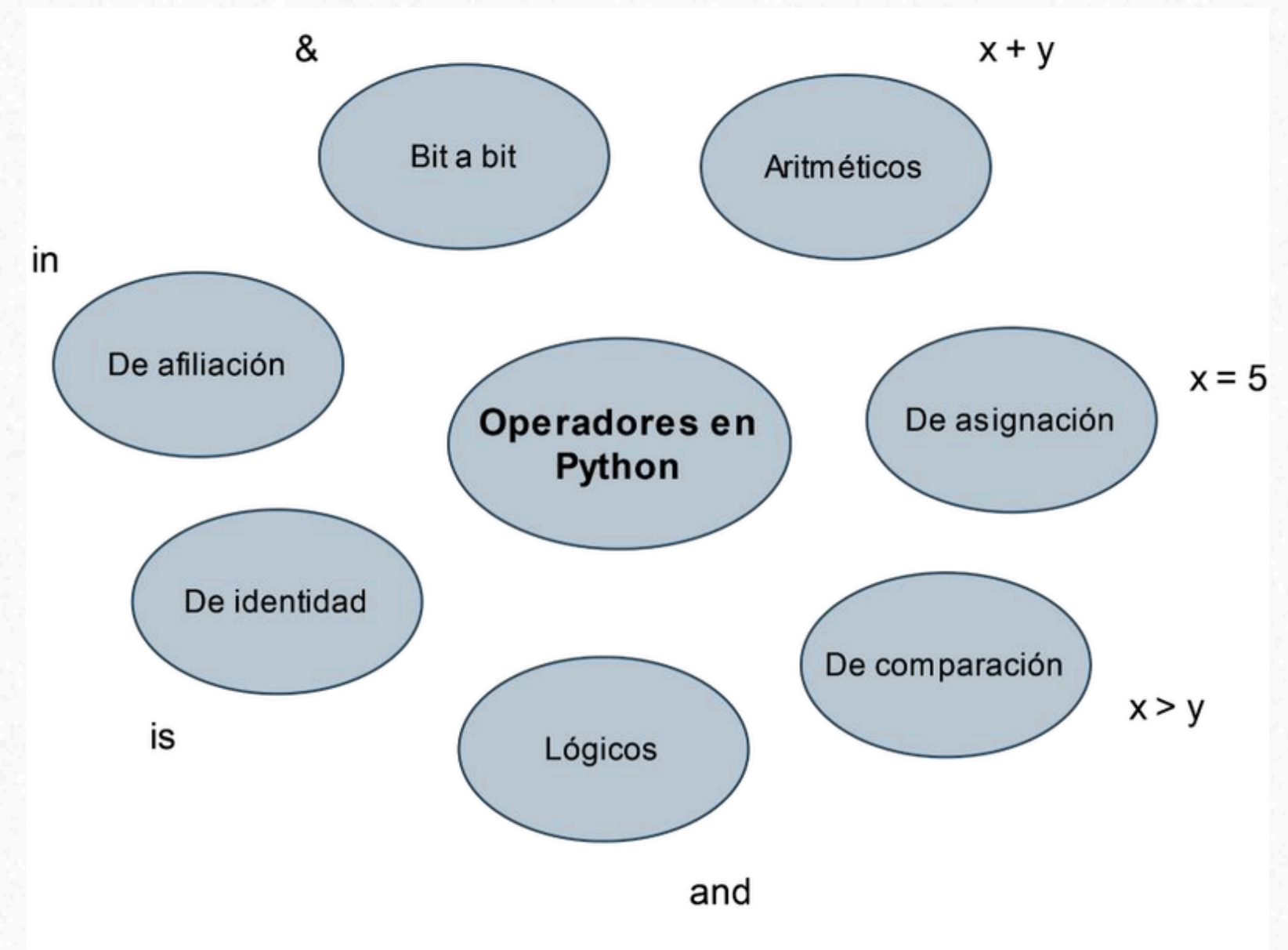
Text Type:	<code>str</code>
Numeric Types:	<code>int</code> , <code>float</code> , <code>complex</code>
Sequence Types:	<code>list</code> , <code>tuple</code> , <code>range</code>
Mapping Type:	<code>dict</code>
Set Types:	<code>set</code> , <code>frozenset</code>
Boolean Type:	<code>bool</code>
Binary Types:	<code>bytes</code> , <code>bytearray</code> , <code>memoryview</code>
None Type:	<code>NoneType</code>

Example	Data Type
<code>x = "Hello World"</code>	<code>str</code>
<code>x = 20</code>	<code>int</code>
<code>x = 20.5</code>	<code>float</code>
<code>x = 1j</code>	<code>complex</code>
<code>x = ["apple", "banana", "cherry"]</code>	<code>list</code>
<code>x = ("apple", "banana", "cherry")</code>	<code>tuple</code>
<code>x = range(6)</code>	<code>range</code>
<code>x = {"name" : "John", "age" : 36}</code>	<code>dict</code>
<code>x = {"apple", "banana", "cherry"}</code>	<code>set</code>
<code>x = frozenset({"apple", "banana", "cherry"})</code>	<code>frozenset</code>
<code>x = True</code>	<code>bool</code>
<code>x = b"Hello"</code>	<code>bytes</code>
<code>x = bytearray(5)</code>	<code>bytearray</code>
<code>x = memoryview(bytes(5))</code>	<code>memoryview</code>
<code>x = None</code>	<code>NoneType</code>

Operadores en Python

Los **operadores** son utilizados para llevar a cabo **operaciones** entre valores o variables.

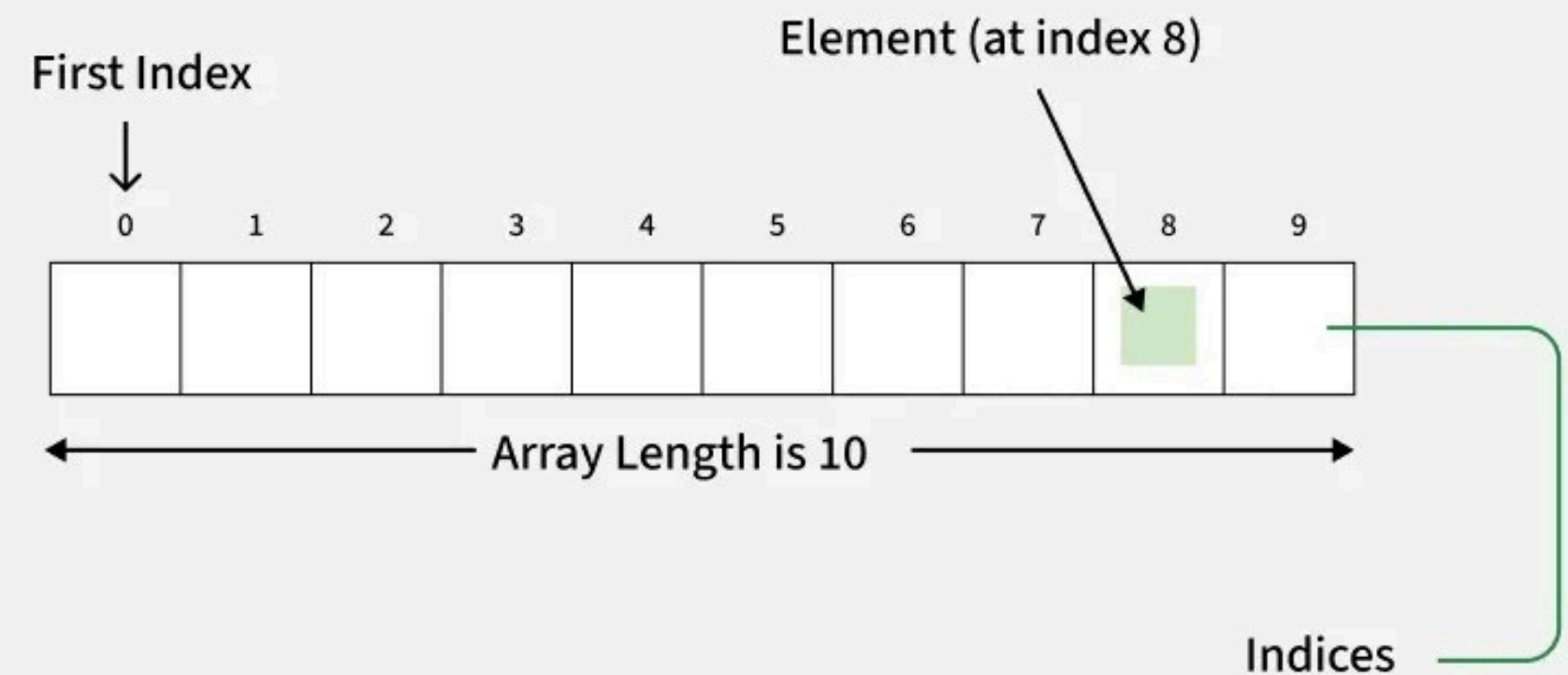
Python divide a los operadores en los siguientes grupos:



¿Qué son los arrays?

Un array es una **variable especial** que puede contener **más de un valor** a la vez.

Un array puede contener muchos valores bajo un **mismo nombre**, y se puede acceder a ellos mediante un número de **índice**.



Indexing

Indexar un array es lo mismo que **acceder** a un elemento del array.

Se puede acceder a un elemento del array consultando su **número de índice**.

Los índices en los arrays de NumPy empiezan por 0, lo que significa que el primer elemento tiene índice 0, el segundo 1, etc.

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[0])
```


Slicing

En Python, slicing significa **tomar elementos** desde un índice dado hasta otro índice dado. Pasamos la **segmentación** en lugar del índice, así: “[inicio:fin]”.

También podemos definir el **paso**, así: “[inicio:fin:paso]”.

Si no especificamos `inicio`, se considera 0.

Si no especificamos `fin`, se considera la longitud del array en esa dimensión.

Si no especificamos `paso`, se considera 1.

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5])
```

Iterating

Iterating significa **recorrer** los elementos **uno por uno**.

Al trabajar con arreglos multidimensionales en NumPy, podemos hacerlo usando un bucle `for` básico de Python.

Si iteramos sobre un arreglo unidimensional, recorreremos cada elemento uno por uno.

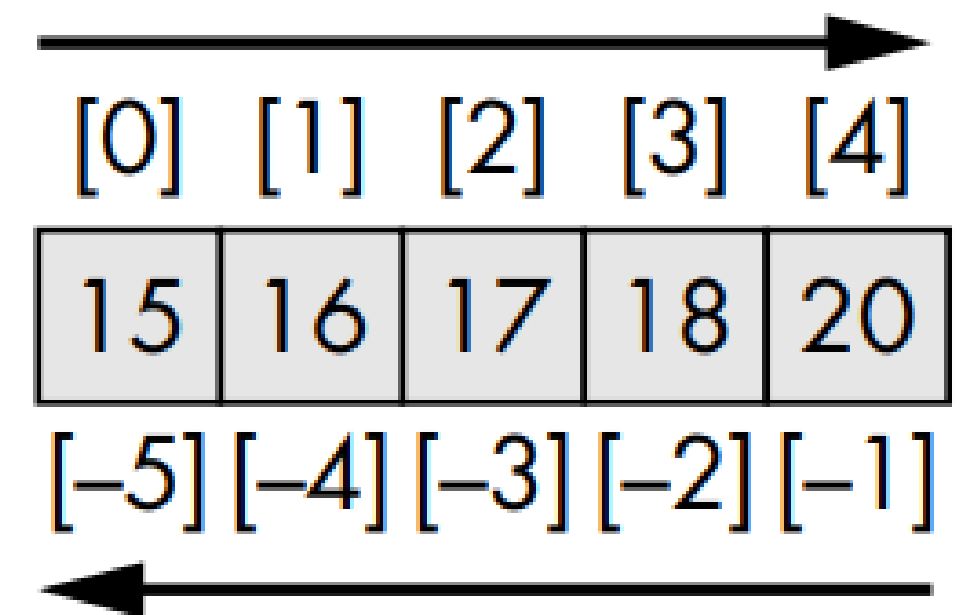
```
import numpy as np

arr = np.array([1, 2, 3])

for x in arr:
    print(x)
```

Trucos de **indexación** en Numpy

- **Indexación booleana:** Selecciona elementos basados en una condición. Se crea una máscara booleana (un arreglo de True/False) donde los True corresponden a los elementos que cumplen la condición.
- **Indexación con arreglos de enteros:** Selecciona elementos en índices no contiguos. Se puede pasar un arreglo de enteros como índice para seleccionar elementos en esas posiciones específicas.



Funciones y métodos de Numpy

**De creación de
arrays**

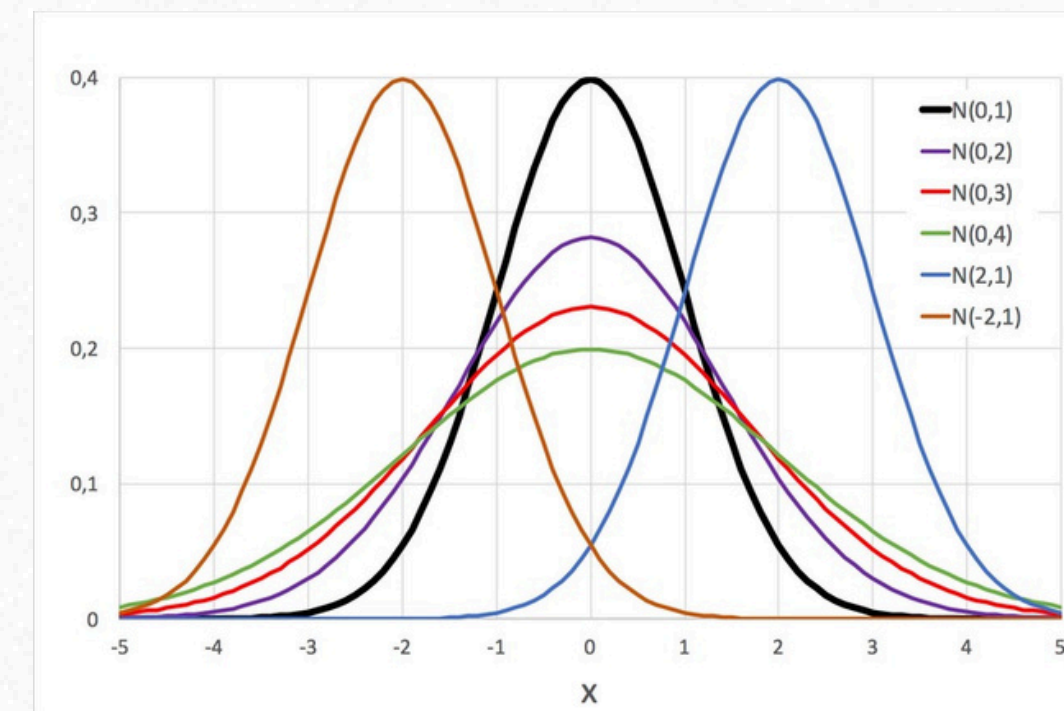
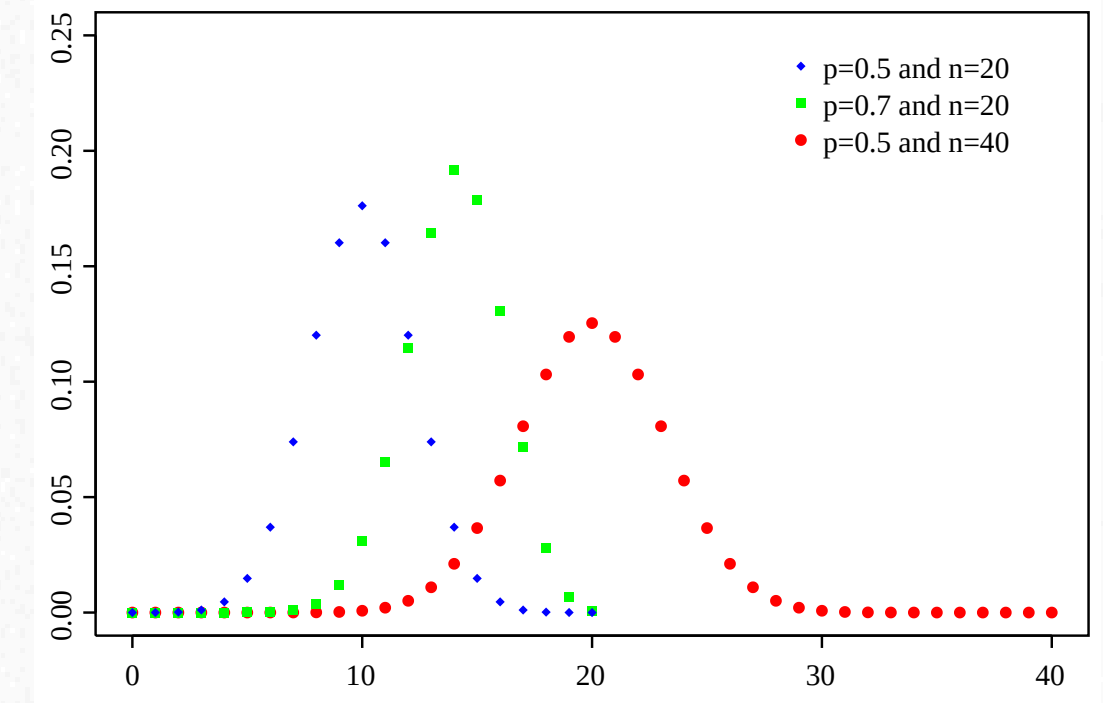
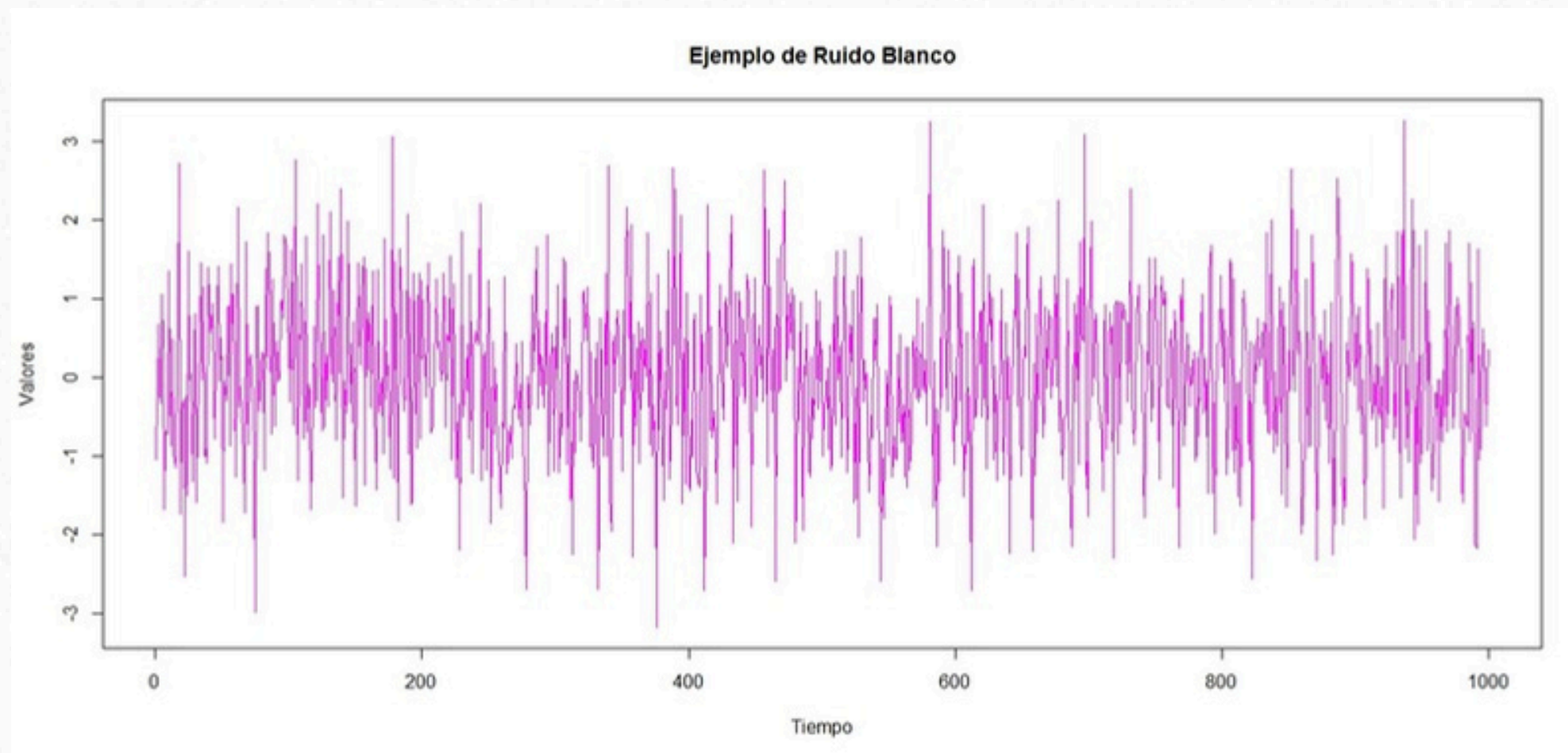
Matemáticos

Estadísticos

**De
manipulación
de arrays**

**De indexación
y selección**

Generación de números aleatorios con Numpy



¡GRACIAS!